

DFMT API

The DFMT API Documentation details the process of installing and setting up the necessary components to run and use the API. This documentation will also aim to provide a brief tutorial regarding how to send requests and interpret the results.

View Demo »

Table of Contents

- 1. About the API/Docs
- 2. Getting Started
  - 2.1. Installation
  - 2.2. Starting the API
  - 2.3. Sending Requests
    - 2.3.1. Postman
    - 2.3.2. Curl
  - 2.4. Authentication
- 3. API Endpoints
  - 3.1. What Is an Endpoint?
- 4. Demo Using Postman
  - 4.1. Creating Catalogs
  - 4.2. Deleting Catalogs
  - 4.3. Listing Catalogs
  - 4.4. Loading Images into a Catalog
  - 4.5. Listing Images Present Within a Catalog
  - 4.6. Querying Files

1. About the API/Docs

The DFMT API is a Non-Graphical tool that implements the functionality of the dfmt executable. It can be used through the terminal or through an API client such as Postman. Some of the features include:

- Create and Delete Catalogs.
- Load **multiple** disk images into a catalog.
  - Prevents loading of same image twice in a catalog.
  - Print out which images have been loaded into a catalog.
- Query **multiple** files against a catalog.

It is important to note that the following terms are used in the documentation:

- **Catalog**: The hash database in which images are loaded into.
- **Image**: The raw disk image.
- **Files**: The media that will be queried (png,pdf,jpg,etc.).

2. Getting Started

To begin using the DFMT API, follow the instructions outlined below to ensure that no errors occur.

2.1. Installation

If using Postman to send API requests, download here.

In order to run the API, you **must** install the dfmt package which contains all the necessary modules/scripts to run. To do this, change directories until you are located in the directory which contains the python 'setup' file and run the following command:

```
# This will install the dfmt package locally onto your computer
python setup.py install
```

2.2. Starting the API

After installing the dfmt package, it contains commands which can **locally** start and stop the API. To run the API, in your terminal, run the following command to start:

```
# Starts the API and sets the API Key
dfmt start --api-key=<ACCESS_TOKEN>
```

Note that in order to send requests, proper authentication is needed. When starting up the API, an API key must be specified using the "**--api-key**" argument followed by the token.

To get more information regarding the dfmt package, run the following command:

```
# Request more information regarding different available commands
dfmt start --help
```

These commands can be entered from any directory as long as the package was correctly installed.

2.3. Sending Requests

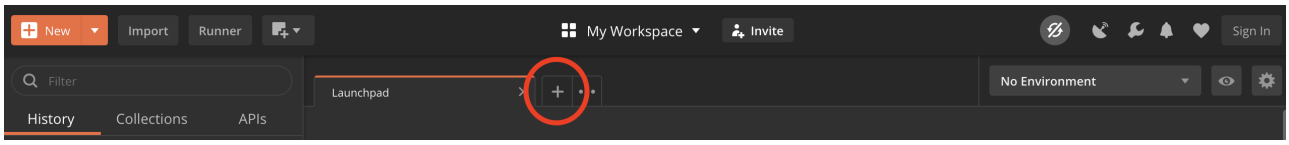
Requests can be sent to the API using:

- Postman
- Curl

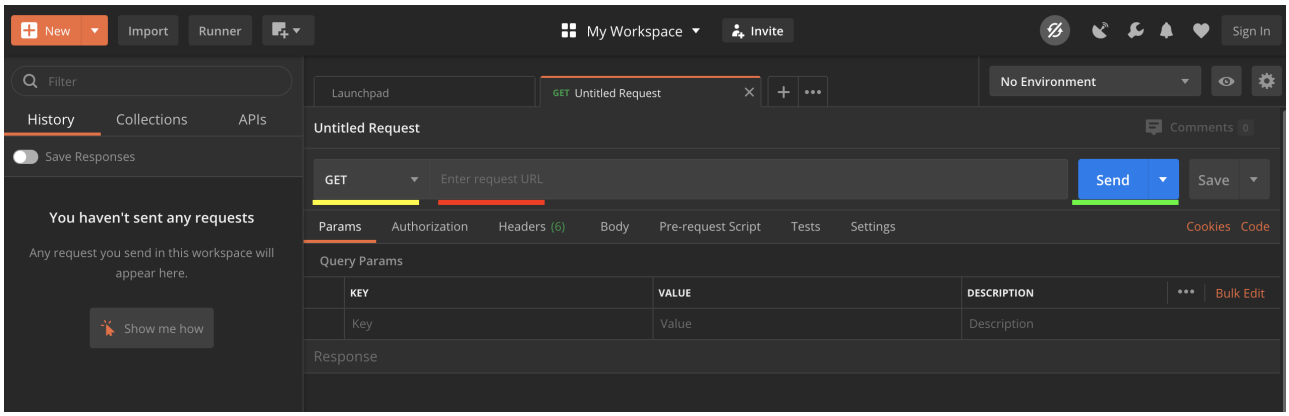
While both methods send the same command to the API, Postman is a GUI based API client that allows for a visual way of testing the different endpoints available. Curl, however, is a command that is used in the terminal to transfer/recieve data from servers and API's.

2.3.1. Postman

1. Make sure that Postman is downloaded and installed.
2. Start the API as mentioned in the 'Starting the API' section.
3. After opening up Postman, click on the highlighted button below to create a new request:



4. Before a request can be made, proper authentication is required. See the authentication section for more information.
5. Now that the API is running, you can now begin sending requests.
    - The yellow line represents the type of request that can be sent: **GET**, **POST**, and **DELETE** are available in the API.
    - The red line is where the request is entered.
    - The green line sends the request to the API.



6. You can find a demo with different requests being sent to the API in the demo section. The demo also explains how to format the body of the request such as uploading images to the catalog and querying files.

2.3.2. Curl

1. Start the API as mentioned in the 'Starting the API' section. After starting, you should open up a new terminal screen so that the curl commands can be inputted.
2. The general syntax to 'curl' a request to the API is as follows:

```
# Curl followed by options then the url to be
curl [options] [url]
```

3. Before a request can be made, proper authentication is required. See the authentication section for more information.

4. To format a curl command, view the 'API Endpoints' section that gives you the associated curl command based on the action you want to take.

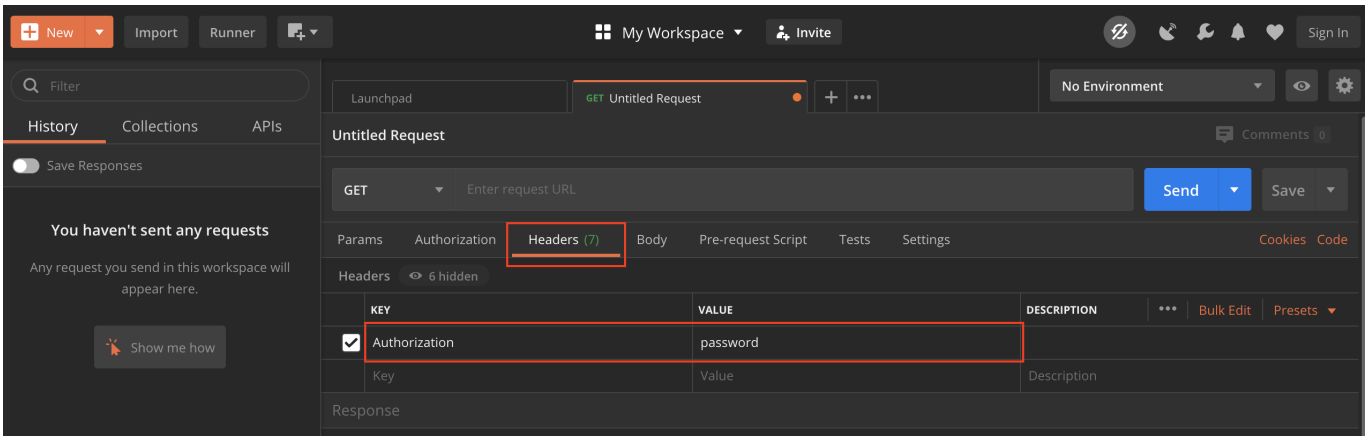
2.4. Authentication

In order to prevent unauthorized users from sending requests, all requests to the API **require** an auth token/password. The authorization token is created when starting up the API and can be found in the 'Starting the API' section.

2.4.1. Using Postman

Providing the password/token in the request is quite simple. Using Postman:

1. Click on the **Headers** tab.
2. Set the key to **Authorization**.
3. Change the value to the specified authorization token. Note that this only needs to be done once if using Postman.



2.4.2. Using Curl

Using Curl, the Authorization header can be provided as follows:

```
# Where <ACCESS_TOKEN> is replaced with the specified password
curl -H "Authorization: <ACCESS_TOKEN>" http://www.example.com
```

2.4.3. Invalid Authentication

If an incorrect password is sent along with the request, the following output body is returned:

```
{
  "status": 401, // HTTP 401 Unauthorized Error
  "message": "Invalid Authentication Credentials"
}
```

3. API Endpoints

This section of the documentation will outline the different requests that can be sent to the API along with their requirements such as parameters and files. Aditionally, this section will detail the responses from the API and how to interpret them.

3.1. What Is an Endpoint?

An API Endpoint is simply the communication channel through which an API interacts with. In our case, it is a URL that contains information in the background that is then sent to the API to be processed. After the request is processed, the output (*JSON*) is sent back to the user who made the request.

Shown below is a request that can be sent to the API:

```
# Finds the specified catalog and returns all images that are loaded
inside of it.
https://localhost:8000/catalogs/{catalog-name}/images # GET request
```

In the example shown above, the request can be classified as a **GET** request since it is attempting to recieve information, not update or delete anything. A **POST** request is used for sending data that should be stored while a **DELETE** request is data that should be deleted from a storage/database.

4. Demo Using Postman

This section will step through different scenarios and go through each one so that the functionality of the API is understood. The demo assumes that the API is setup/installed correctly and is **running**.

*For the sake of simplicity and length, the demo will use **Postman**.* A demo using curl is not provided as the 'API Endpoints' section contains the curl commands needed to mimic the actions shown below.

Each section of the demo will be laid out as followed:

1. Description of the task.
2. How to send the request to the API with the correct formatting.
3. Potential API responses including successes, errors, and improper formatting.

4.1. Creating Catalogs

"Creates an empty catalog with the specified name."

1. Change the request type to **POST**.
2. Type in the following request in the text box:

```
# Where 'tut1' is the name of the catalog to be created
http://localhost:8000/catalogs/tut1
```

3. Ensure that you have followed the authentication steps.

Note that the last portion of the request is the name of the catalog to be created. If the catalog was **successfully created**, then the following output is returned:

```
{
  "status": 200,
  "message": "Successfully created the catalog 'tut1'"
}
```

If 'demodb' already exists, then attempting to create it again with the same identifier will **fail** and output:

```
{
  "status": 400,
  "message": "The catalog 'tut1' already exists"
}
```

4.2. Deleting Catalogs

"Deletes the catalog with the specified name."

1. Change the request type to **DELETE**.
2. Type in the following request in the text box:

```
# Where 'demodb' is the name of the catalog to be removed
http://localhost:8000/catalogs/demodb
```

3. Ensure that you have followed the authentication steps.

If the catalog was **successfully deleted**, then the following output is returned:

```
{
  "status": 200,
  "message": "Successfully removed the catalog 'demodb'"
}
```

If 'demodb' does not exist, then attempting to delete it will **fail** and output:

```
{
  "status": 404,
  "message": "Catalog 'demodb' does not exist"
}
```

4.3. Listing Catalogs

"Lists all non-deleted catalogs that are present within the API."

1. Change the request type to **GET**.
2. Type in the following request in the text box:

```
# Gets all catalogs present within the API
http://localhost:8000/catalogs/
```

3. Ensure that you have followed the authentication steps.

The following request output is returned:

```
{
  "status": 200,
  "data": {
      "catalogs": [
          'demodb',
          'example1',
          'example2'
      ]
  }
}
```

4.4. Loading Images into a Catalog

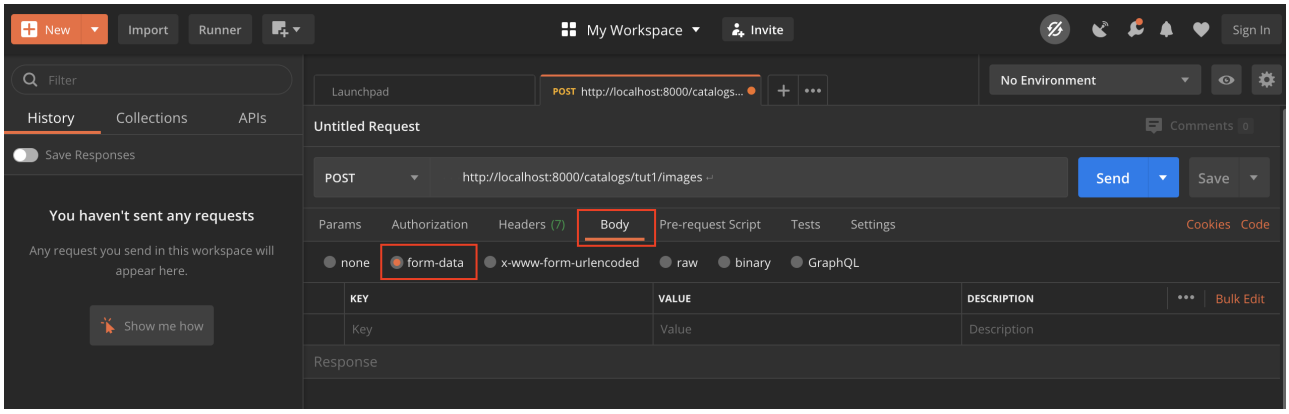
"Loads images into the specified catalog/database."

1. If the catalog does not exist, make sure that one is created. In this example, it will be assumed that the catalog 'tut1' is already created and is initially empty.
2. Change the request type to **POST**.
3. Type in the following request in the text box:

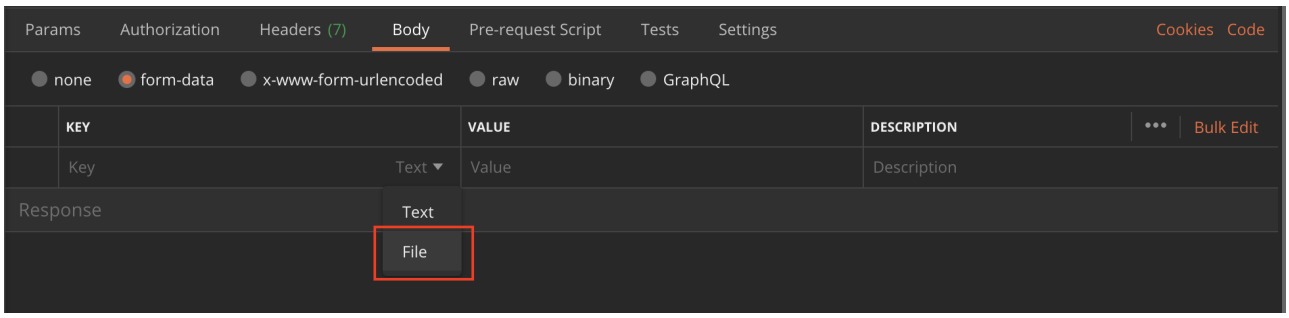
```
# Loads the images provided in the payload into 'tut1'
http://localhost:8000/catalogs/tut1/images
```

4. Ensure that you have followed the authentication steps.

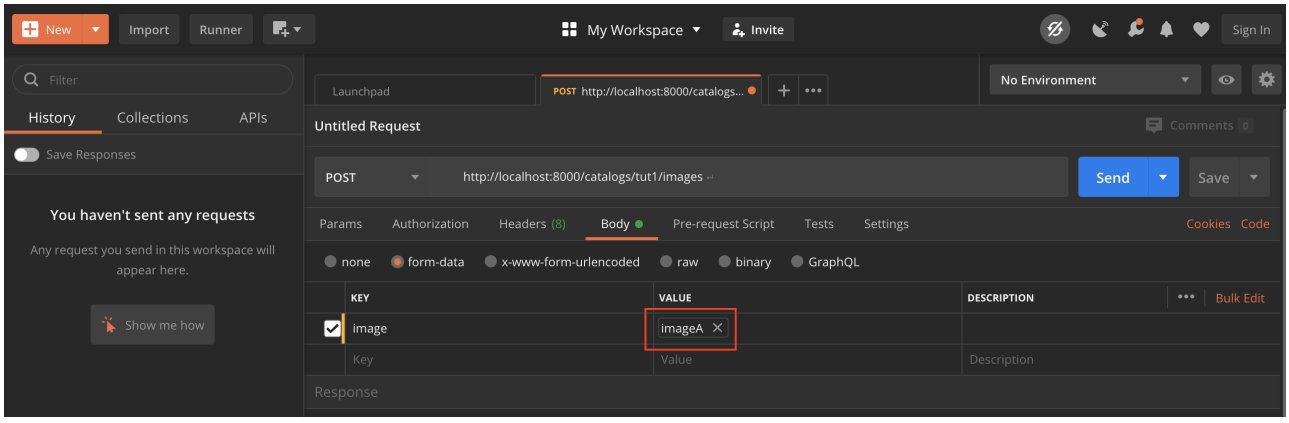
5. Click on the **Body** tab then click '**form-data**':



6. Change the input type to '**File**':



7. Change the key to '**image**'.
8. Click '**Select Files**', then select the image that is to be loaded into the catalog/database. In the example, the image '*imageA*' is being loaded into the catalog:



9. **Repeat** steps 6-8 in a new row for any additional images that are to be loaded into the catalog.

If the image(s) were **successfully** loaded into the catalog, then the following output is returned:

```
{
  "status": 200,
  "data": {
      "catalog": 'tut1', // The name of the catalog
      "images": [
          {
              "image": "imageA", // Image name
              "loaded": true, // Load status
              "message": "Successfully loaded image"
          }
      ]
```

```
    }
}
```

For example, lets say that we loaded 2 images into the catalog: 'imageA' and 'imageB'. If imageA was **previously loaded** into the catalog but imageB was successfully loaded, then the following output is returned:

```
{
  "status": 200,
  "data": {
      "catalog": 'tut1', // The name of the catalog
      "images": [
          {
            "image": "imageA",
            "loaded": false, // Load status
            "message": "Image already loaded"
          },
          {
            "image": "imageB",
            "loaded": true, // Load status
            "message": "Successfully loaded image"
          }
      ]
  }
}
```

4.5. Listing Images Present Within a Catalog

"Lists the images that are loaded inside of a catalog."

1. If the catalog does not exist, make sure that one is created. In this example, it will be assumed that the catalog 'tut1' is already created and **contains** multiple loaded images inside of it.
2. Change the request type to **GET**.
3. Type in the following request in the text box:

   ```
   # Lists images present with in the 'tut1' catalog
   http://localhost:8000/catalogs/tut1/images
   ```

4. Ensure that you have followed the authentication steps.

The following output body is returned:

```
{
  "status": 200,
  "data": {
      "catalog": 'tut1', // The name of the catalog
      "images": [ // The loaded images
          "imageA",
```

```
            "imageB",
            "imageC"
        ]
    }
}
```

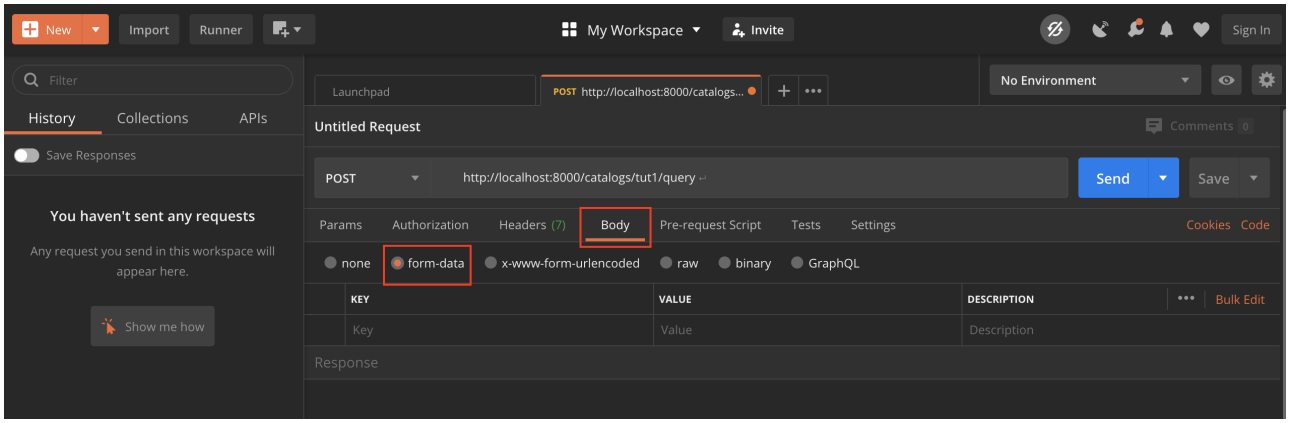
4.6. Querying Files

"Queries file(s) against a specified catalog. Returns a score based on different compute methods."

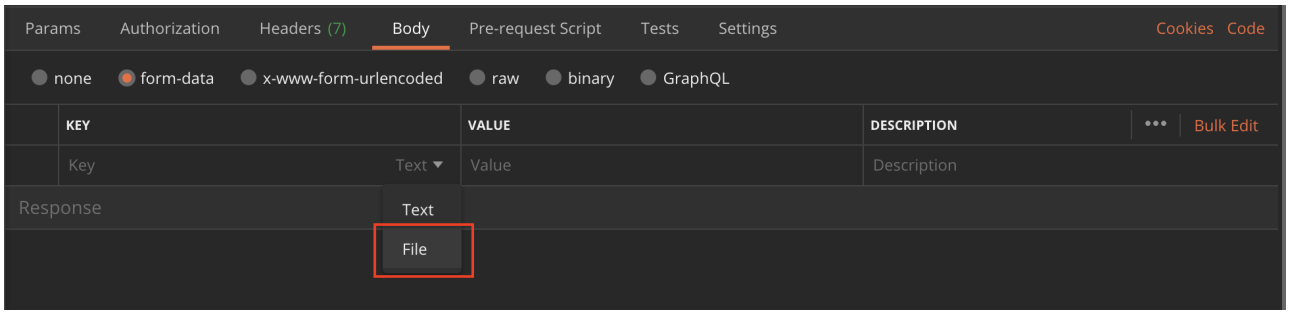
1. If the catalog does not exist, make sure that one is created. In this example, it will be assumed that the catalog 'tut1' is already created and **contains** loaded image(s) inside of it.
2. Change the request type to **POST**.
3. Type in the following request in the text box:

```
# Queries file(s) against a specified catalog
http://localhost:8000/catalogs/tut1/query
```

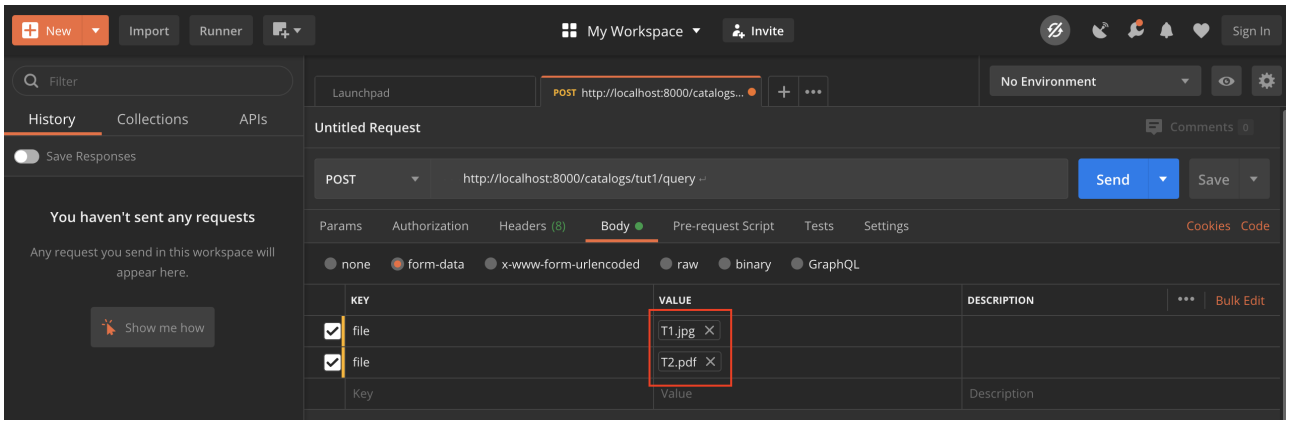
4. Ensure that you have followed the authentication steps.
5. Click on the **Body** tab then click '**form-data**':



6. Change the input type to '**File**':



7. Change the key to '**file**'.
8. Click '**Select Files**', then select the files that should be queried against the catalog. In this example, '*T1.jpg*' and '*T2.pdf*' are being queried against the 'tut1' catalog:



9. **Repeat** steps 6-8 in a new row for any **additional** files that should be queried against the catalog.

After sending out the request, the following output is returned:


```
{
  "status": 200,
  "catalog": 'tut1',
  "data": {
     "T1.jpg": {
          "compute3": {
              "images": {
                  "imageA": {
                     "score": 0.46894397
                  },
                  "imageB": {
                     "score": 0.14849821
                  }
            }
            "unique matched hashes": 260,
            "total sectors in media": 528,
            "compute2": 0.4476010
          }
     }
     "T2.pdf": {
         "compute3": {
             "images": {
                  "imageC": {
                     "score": 0.8241308793
                  },
                  "imageB": {
                     "score": 0.1736937618
                  }
             }
             "unique matched hashes": 575,
             "total sectors in media": 575,
             "compute2": 0.70008695
         }
     }
  }
}
```

Since the request did not specify any parameters, the **default** compute method that is used in the above example is '**compute3**'.

Here are some other request examples that specify which compute method to use:

```
# Uses compute1 only
http://localhost:8000/catalogs/tut1/query?compute1=true&compute3=false
```

```
# Uses both compute3 and compute1
http://localhost:8000/catalogs/tut1/query?compute3=true&compute1=true
```

These parameters can also be added in Postman by:

1. Clicking on the '**Params**' tab.
2. Changing the key to either '**compute3**' or '**compute1**'
3. Setting the value of the key to either '**true**' or '**false**'